

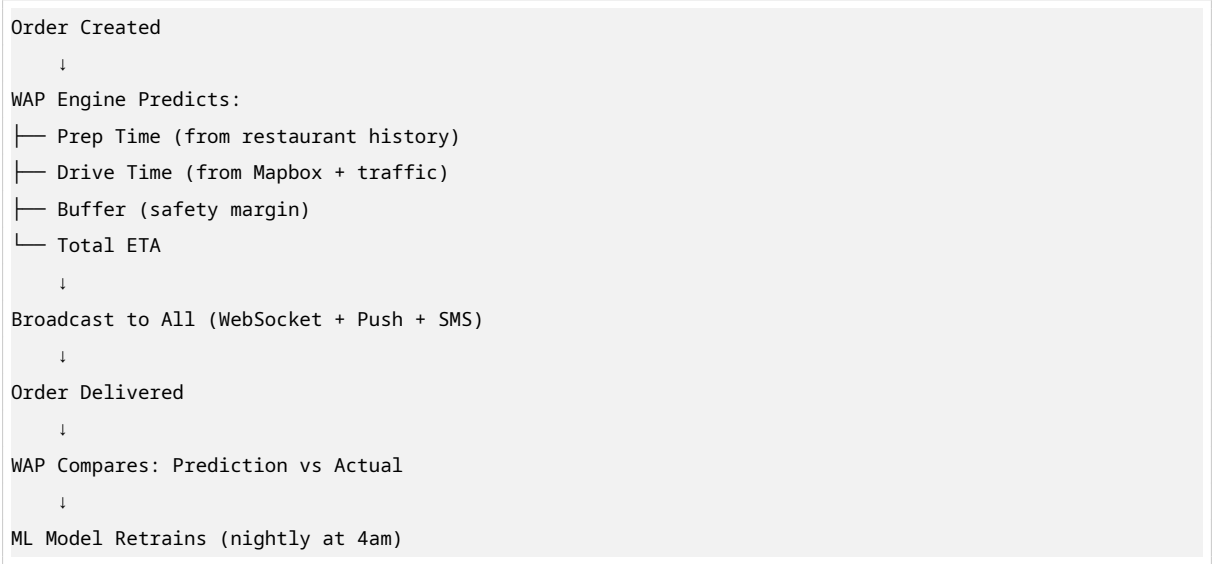
WAP — Wolfie AI Prediction Time System

Overview

WAP predicts order completion time using machine learning and real-time data. Every stakeholder gets accurate ETAs:

Stakeholder	What They See
Customer	“Your order arrives in 25 min”
Restaurant	“Prep time: 12 min —driver arrives at 2:30”
Driver	“Pickup in 12 min, delivery in 13 min”
Wolfie	Confidence score + matching optimization

Architecture



Prediction Components

Component	Weight	Source
Prep Time	40%	Restaurant historical data
Drive Time	35%	Mapbox routing + traffic
Buffer	25%	Variance + weather + rush hour

Confidence Score

Confidence	Meaning	Action
90-100%	Very sure	Auto-dispatch
70-89%	Confident	Standard dispatch
50-69%	Uncertain	Add buffer time
<50%	New restaurant	Use defaults

Machine Learning

Features:

- Item count
- Order complexity
- Rush hour (yes/no)
- Day of week
- Hour of day
- Restaurant historical variance

Training:

- Retrain every 10 new orders
- Nightly full retrain at 4am
- Compare: MAE, RMSE, MAPE

Feedback Loop:

```
Predicted: 25 min
Actual: 28 min
Error: +3 min (+12%)
→ Model adjusts for next prediction
```

Files

File	Purpose
services/wap.py	WAP Engine (prediction + feedback)
database/schemas_wap.py	Database tables
hooks_wap.py	Order state hooks
tasks/wap.py	Celery tasks (notifications + retrain)

API Integration

Prediction stored in order:

```
{
  "order_id": "abc-123",
  "eta_predicted": 25.5,
  "prediction_data": {
    "prep_time_min": 12.0,
    "drive_time_min": 10.5,
    "buffer_min": 3.0,
    "confidence": 0.85,
    "breakdown": { ".|.|" }
  }
}
```

Real-time updates via WebSocket:

```
socket.on("eta_update", (data) => {
```

```
console.log(data.eta_min); // 25
});
```

Installation

```
# Add to requirements.txt
scikit-learn>=1.3.0
numpy>=1.24.0

# Register hooks in app.py
from hooks_wap import register_wap_hooks
register_wap_hooks()

# Add to Celery beat schedule
"wap-retrain-nightly": {
    "task": "tasks.wap.retrain_all_models",
    "schedule": crontab(hour=4, minute=0),
}
```